The Complete Explainer

Self-Supervised AI for ETL Error Detection and Root-Cause Localization

- Part 1: What the project is
  - The problem, in one paragraph
  - What this project does about it
  - Why the second question is the hard one
  - Why it has to work without labels
- Part 2: The concepts, defined
  - ETL
  - Task
  - DAG
  - Batch, or run
  - Profile
  - Self-supervised learning
  - Masking
  - Anomaly
  - Ancestors and descendants
  - Calibration
  - Ablation
- Part 3: The benchmark
  - Why one had to be built
  - How the simulation works
  - Faults are injected into the rows, never into the numbers
  - The fifteen fault types
  - Measured propagation, not assumed
  - The dataset
- Part 4: The model
  - The shape of it
  - Attention in two directions
  - The DAG bias

    - Calendar features
    - Training
- Part 5: How it answers the three questions
    - Scoring mirrors training
    - Is the run broken?
    - Which task caused it?
    - What kind of break is it?
- Part 6: The results
    - The headline
    - The finding that survived
    - Calibration is not always good
    - Ablations: four null results
    - The surprise: training longer makes it worse
    - Explainability
    - Where it fails
- Part 7: Evaluation, and why it is set up this way
    - Chronological splits
    - Contaminated training history
    - Robust normalisation
    - The metrics
    - Point-adjusted F1 is reported but never headlined
- Part 8: How to reproduce everything
    - What lives where
- Part 9: Mistakes made, and what they cost
    - The peeking bug in the culprit-finder
    - An unfair comparison, then a public correction
    - Timestamps a thousand times too small
    - Bars that rendered empty
    - The lesson in all four

Everything in this project, explained once, properly. If something in the code, the report or the slides does not make sense, the answer is here.

Written to be read by someone who has not seen the code. Jargon is defined the first time it appears.

Part 1: What the project is

The problem, in one paragraph

Companies move data around with **pipelines**: pull records out of the systems that create them, clean them up, join them together, load them somewhere people can run reports on. These pipelines run on a

schedule, often every hour. They break in ways that do not crash anything. A price field switches from rupees to paise upstream, and the pipeline happily loads numbers that are a hundred times too big. Nobody notices for weeks, because nothing failed. It gets found by a human squinting at a dashboard.

What this project does about it

Two things, for every pipeline run:

1. **Detection.** Is this run broken?
2. **Root-cause localization.** Which task broke it, and what kind of break is it?

Why the second question is the hard one

When one task breaks, every task downstream of it also looks broken. The cleaning step passes on the nulls, the join loses rows, the aggregate under-reports revenue, the warehouse load comes out thin.

Worse, the damage often **grows** as it travels. A five percent null rate at extraction can become a thirty percent revenue drop after aggregation. So the loudest alarm is frequently the furthest from the actual problem.

A detector that flags nine unhealthy tasks has told an on-call engineer nothing they did not already know.

Why it has to work without labels

Labelled examples of pipeline failures barely exist. Failures are rare, recorded inconsistently when recorded at all, and the ones nobody noticed are still sitting in the history unmarked. A method that needs a thousand labelled examples of schema drift is a method nobody can deploy.

So the model is trained **self-supervised**, which means it learns from data that carries no answers.

Part 2: The concepts, defined

ETL

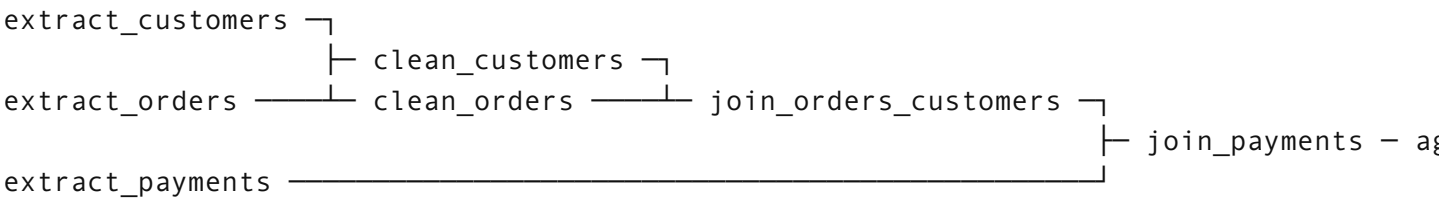
Extract, Transform, Load. The three stages of moving data: get it out of the source system, reshape it, put it into the destination.

Task

One step in a pipeline. “Pull today’s orders” is a task. “Join orders to customers” is a task. This project simulates nine of them.

DAG

Directed Acyclic Graph. The map of which task feeds which. Directed because data flows one way. Acyclic because there are no loops. It is simply the shape of the pipeline.



Batch, or run

One execution of the whole pipeline, covering one hour of data. The dataset has 8000 of them, which is about eleven months.

Profile

The 38 numbers that summarise what one task did in one run. Not the data itself, a summary of it. Things like: how many rows came out, what fraction of fields were empty, how many duplicate keys, how far behind the timestamps are, how long the task took, how many keys failed to match.

A whole run is therefore nine profiles, one per task, which is a 9 by 38 grid of numbers. The model looks at a window of the last 32 runs, so its input is 32 by 9 by 38.

Self-supervised learning

Learning without answer labels, by hiding part of the input and predicting the hidden part. The rest of the input serves as the answer key.

The everyday version: cover a word in a sentence and guess it. To be good at that you have to learn how language works. Same idea here. Cover part of a pipeline run, predict it, and to be good at that you have to learn how the pipeline normally behaves.

Masking

The hiding step. This project hides two ways:

- **Feature-group masking.** Hide one family of measurements at one task, for example all five “how empty is it” numbers.
- **Task masking.** Hide a task’s entire profile, so the model has to work out what that task should have produced from what its inputs looked like.

Anomaly

Something the model cannot predict. Once it is good at filling in hidden parts of normal runs, whatever it fails to fill in is unusual.

Ancestors and descendants

A task's ancestors are everything upstream that feeds it, directly or indirectly. Its descendants are everything downstream that it feeds. For `join_orders_customers`, the ancestors are the two clean steps and the two extract steps above them, and the descendants are the payments join, the aggregate and the warehouse load.

Calibration

Putting scores from different tasks on the same footing. Some tasks are naturally harder to predict than others, so their raw errors are not comparable, and the task that is always hardest wins any ranking by default. Calibration converts each task's error into "how unusual is this **for this task**", by comparing it against that task's own history.

Ablation

Removing one piece of a design, retraining, and measuring whether anything got worse. If nothing changes, that piece was not doing anything.

Part 3: The benchmark

Why one had to be built

No public dataset labels which pipeline task caused a failure.

The datasets everyone in this field uses (called SMD, SWaT, SMAP, MSL) are server and sensor readings. They have no pipeline structure, so the question "which step caused this" cannot even be asked of them. Meanwhile the systems research that does work on real pipeline data uses company data that is never released.

So the benchmark is part of the contribution, not a shortcut around a missing one. This is the point most likely to be challenged, and it is worth being firm about.

How the simulation works

Nine tasks, each performing a genuine transformation on genuine dataframe rows: type casting, deduplication, currency conversion to a common unit, left joins, group-by aggregation, and a final publish step.

Source data comes from a persistent pool of 20,000 customers, so foreign keys point at something real. Volume follows a daily cycle, a weekly cycle, a growth trend, slow drift in the customer country mix, and occasional marketing campaigns that raise volume by about 1.7x for a day or two.

All of that variation is labelled normal on purpose. Without it, a simple threshold on row count would score well and no deep model would be justifiable.

Faults are injected into the rows, never into the numbers

This is the most important design decision in the whole project.

If a fault were injected into the summary numbers directly, somebody would have had to decide by hand how the damage spreads to the next task. The model would then be learning those hand-written rules rather than anything about data pipelines. That is circular reasoning.

By corrupting the actual rows, the next task genuinely reads broken data, and the damage spreads by itself as a consequence of the real transformations.

The fifteen fault types

Family	Faults	What happens
Schema	`schema_column_drop`	A column silently disappears
	`schema_column_add`	An unannounced column appears
	`type_mismatch`	Numbers arrive as text, casts fail, values become empty
Completeness	`null_spike`	A source starts emitting empty fields
	`pk_null`	Record IDs go missing, breaking deduplication
	`duplicate_records`	A re-delivered batch duplicates rows
	`row_loss_truncation`	A partial load cuts the batch short
Value	`unit_change`	Currency or unit convention changes, e.g. rupees to paise
	`value_corruption`	Extreme or sentinel values appear
	`category_corruption`	Encoding drift mangles category labels

Family	Faults	What happens
Temporal	`timezone_shift`	A producer switches from UTC to local time
	`late_arrival`	The source lags, carrying stale events
Integrity	`fk_break`	Key formatting drifts, so joins orphan rows
	`join_fanout`	Duplicate keys turn a one-to-one join into one-to-many
Operational	`resource_degradation`	The task blows its deadline, output stays valid

`resource_degradation` is deliberately awkward. The task misses its schedule while its data is perfectly correct, so a detector that only looks at data quality must not confuse it with corruption, and it must not spread downstream.

Measured propagation, not assumed

The obvious way to label which tasks an incident affected is: the origin plus every descendant. That is wrong, and provably.

- An unexpected extra column is ignored by every downstream select.
- A task that runs slowly still emits correct rows.

Labelling those descendants anomalous would penalise a detector for being right about them.

So the simulator runs **every faulty batch a second time with the fault removed**, holding the source rows, the telemetry noise and the scheduling jitter identical. This works because each task draws its randomness from its own generator seeded on `(seed, batch, task)`, which guarantees the two runs are bit-identical wherever the fault does not reach. A test asserts exactly that.

A task counts as affected only when its numbers actually moved.

	Tasks affected per incident
Measured	2.48 of 9
Assumed (origin plus descendants)	3.94 of 9

`schema_column_add` correctly reaches exactly one task, where the assumption claims 4.5.

The dataset

Property	Value
Runs	8000 hourly, about eleven months

Property	Value
Shape	8000 x 9 x 38
Incidents	273, spanning 594 runs (7.4%)
Contamination in training history	3.0%
Incident rate in the test period	14.6%
Split	Chronological, 60 / 15 / 25

Part 4: The model

The shape of it

A **transformer**, which is a kind of neural network that looks at all parts of its input at once and learns which parts to pay attention to, rather than reading strictly in order. About 1.1 million parameters, which is small.

Its input is a window of 32 runs, each with 9 tasks, each with 38 numbers.

Attention in two directions

Attending over all 32 by 9 positions at once would be wasteful, because the task axis is short and the time axis is long, and they carry different kinds of structure. So each layer does two separate passes:

1. **Along time.** Each task looks back over its own recent history.
2. **Across the DAG.** Each moment in time looks across the other tasks.

The DAG bias

Every ordered pair of tasks is labelled by relationship: self, direct parent, direct child, distant ancestor, distant descendant, or unrelated. The model learns one attention adjustment per relationship, so it can decide for itself how much a task should listen to its immediate upstream.

This turned out not to help. See Part 6.

Calendar features

Hour of day and day of week, encoded as sine and cosine pairs so that 23:00 and 00:00 are close together.

This also turned out not to help. See Part 6.

Training

Hide part of the input, predict it, measure the error only on the hidden parts. Predicting the visible parts would be free.

Masking whole groups rather than scattered numbers matters, because `null_rate_mean` sits next to `null_rate_max` in the same vector and is trivially recoverable from it. The model could learn within-vector shortcuts and never look at the pipeline. Hiding an entire group closes that off.

Checkpoint selection uses no labels. The saved model is the one with the best validation reconstruction loss, which needs no annotations. Choosing it by detection accuracy instead would quietly make the method weakly supervised and inflate every number afterwards.

Part 5: How it answers the three questions

Scoring mirrors training

The model was trained to fill in hidden entries, so at inference it is asked the same kind of question. Running a masked model **unmasked** lets it copy its input straight to its output, and the reconstruction error then measures nothing at all.

Is the run broken?

Hide the entire final run and ask the model to reconstruct it from history alone. Whatever it cannot reconstruct is the surprise. Take the largest surprise across the nine tasks; that is the run's anomaly score.

Which task caused it?

For each task in turn, hide **that task and everything downstream of it**, then ask the model to predict what that task should have produced. Its ancestors and its own history stay visible.

- If the prediction is close, the task's output follows sensibly from its input. It is a **victim**.
- If the prediction is far off, the task produced something strange even though its inputs were fine. That is the **origin**.

The bug that hiding descendants fixes

The first version hid only the task being tested. That leaks. Attention across the DAG runs in both directions, so the model could still see that task's own descendants, which during an incident are also broken, and work backwards from them to the corrupted value. It was reading the answer.

Hiding the descendants as well took accuracy from 63.2% to 77.3%.

The proof that this is the real mechanism rather than a coincidence is in the depth breakdown. The leak should hurt most where a task has the most descendants, and vanish entirely at the last task, which has none. It does exactly that: at the final task all three scoring rules give an identical 0.893, because the three masks are then the same tensor.

What kind of break is it?

The per-feature prediction error at the identified task, grouped into the eight named families: operational, schema, completeness, metric distribution, numeric distribution, categorical distribution, temporal, integrity.

Because the simulator knows which family each fault type should move, this can be **scored** rather than merely displayed.

Part 6: The results

The headline

Test split: the last 25% of the timeline, 2000 runs, 291 carrying an incident. Nothing was tuned on test.

Method	AUPRC	Hit@1	Hit@3	MRR
LSTM autoencoder	0.951	**0.818**	**0.986**	**0.900**
PCA residual	0.942	0.801	0.955	0.880
This model	0.953	0.797	0.973	0.883
Local Outlier Factor	**0.973**	0.773	0.976	0.872
Dense autoencoder	0.956	0.749	0.924	0.843
Rolling z-score	0.502	0.570	0.942	0.748
Isolation Forest	0.466	0.340	0.701	0.550

The model does not win, on either task. Say so before you are asked.

With 291 incidents the standard error on a Hit@1 near 0.8 is 0.023, and the seed-to-seed variation of this model is 0.011, so the top four are tied rather than ranked.

Why: each task is described by only 38 numbers and there are nearly 5000 clean training examples, which is close to ideal for simple statistical methods. Five of the seven methods sit within 0.03 AUPRC of each other.

The finding that survived

Ranking nine raw errors asks which task has the biggest error. The question is which task is furthest from **its own** normal. Two independent ways to fix that turn out to be worth the same:

Scoring rule	Hit@1 [95% CI]
Rank by raw error	0.588 [0.533, 0.646]
Compare each task to its own history (**statistical**)	0.773 [0.725, 0.821]
Ask if it is surprising given its inputs (**structural**)	0.773 [0.725, 0.821]
Both	0.797 [0.749, 0.842]

Rows two and three are identical to three decimal places with the same interval. Doing both adds 2.4 points, which the intervals cannot separate from noise.

That 18.5 point gain is larger than the entire spread between the seven methods, which is about 4 points. **How you score matters more than which model you use.**

A correction

An earlier version of this work reported the structural fix as worth 18.5 points on its own. That compared it against an uncalibrated baseline, which was not a fair fight. Corrected, it is worth 2.4 points over the statistical fix.

Calibration is not always good

Method	Uncalibrated	Calibrated	Change
LSTM autoencoder	0.643	0.818	**+0.175**
Dense autoencoder	0.725	0.749	+0.024
This model	0.773	0.797	+0.024
PCA residual	0.801	0.801	0.000
Local Outlier Factor	0.797	0.773	-0.024
Rolling z-score	0.663	0.570	**-0.093**

It transforms the LSTM autoencoder and damages the rolling z-score. The pattern follows how far apart a method's per-task error scales already are. The LSTM reconstructs all nine tasks through one shared hidden state, so its per-task scales differ wildly and there is a lot to correct. PCA and LOF already produce comparable per-task scores, so the rank transform only throws away magnitude.

Ablations: four null results

Identical budget for every configuration, 40 epochs with the same early-stopping patience. Three seeds establish the noise floor at Hit@1 0.805 ± 0.011.

Configuration	Hit@1	Change	Verdict
Full model (3 seeds)	0.805	reference	
Clean training set	0.814	+0.009	within noise

Configuration	Hit@1	Change	Verdict
No DAG attention bias	0.801	-0.005	within noise
No calendar features	0.790	-0.015	within noise
Working calendar features	0.787	-0.018	within noise
Contrastive objective added	0.787	-0.018	within noise
Longer training (57 epochs)	**0.756**	**-0.049**	**outside noise**

The table has six rows for four ideas, because the calendar appears twice: once as originally measured, and once re-measured after the timestamp bug that made the first attempt vacuous. Both give the same answer.

Each null result has a reason:

The DAG bias is redundant because the model finds the pipeline structure anyway. Without the bias, attention still puts 42.2% of its off-diagonal weight on true pipeline connections against 22.2% expected by chance, compared with 46.4% with the bias. Telling it the structure nudges it and changes nothing.

Calendar features are redundant because a 32-hour window already spans a full day, so the phase is inferable from the visible history.

The contrastive objective costs four times the compute, degrades reconstruction from 0.438 to 0.486, and changes nothing else. Its own loss collapses to 0.055, meaning the task became trivial. The augmentations are weak on purpose, because strong ones would manufacture something indistinguishable from a real fault and teach the model that real faults are normal.

Contamination tolerance is confirmed. Training on only clean windows gives +0.009, inside noise, so the method genuinely tolerates the 3% contamination real history carries.

The surprise: training longer makes it worse

Training 57 epochs instead of 33 improves validation reconstruction from 0.433 to 0.405 and drops localization from 0.805 to 0.756.

This is over-generalisation. The model works by being surprised by broken data. Train it well enough and it becomes good at predicting broken data too, so the surprise shrinks. The contrastive run says the same from the other direction: clearly worse reconstruction, unchanged localization.

This undermines the project's own stopping rule. Checkpoints were chosen on validation reconstruction loss precisely because it needs no labels, and that measure is now shown not to track the thing that matters. It is written up as an open problem rather than hidden.

Explainability

Method	Accuracy over 291 incidents
Integrated Gradients	**0.935**
Reconstruction error	0.924

Both high. Integrated Gradients is clearly better on `timezone_shift` (0.95 against 0.76), which fits: it can credit evidence in other tasks and earlier runs, while reconstruction error only sees the flagged task at the flagged moment. A timestamp shift shows up as a relationship across the window rather than as one large residual.

Since it costs 32 extra forward and backward passes for a one-point average gain, reconstruction error is the sensible default.

The attention recovered the pipeline. Averaged over incidents, structural attention places 46.4% of its weight on true pipeline connections against 22.2% expected by chance, a 2.1x lift. Nothing forces this; the DAG bias is a learned number per relationship, not a hard rule.

Where it fails

Fault type	Hit@1	Tasks it reaches
`row_loss_truncation`	0.29	3.77
`duplicate_records`	0.43	2.84
Nine other types	0.94 to 1.00	1.00 to 1.76

The correlation between how far a fault spreads and how well it localizes is **-0.63**. Faults that stay local are easy; faults that travel are hard.

The pattern is sharper than the average suggests. Of the mislocalized `duplicate_records` incidents, **100%** blamed a task further downstream, nearly always the same join.

The reason is specific. A join's output row count depends on how well the keys overlap between its two inputs, and key overlap is not among the 38 measurements. No model can predict a join's row count from its parents' profiles, so when a duplication arrives from upstream the fan-out it causes at the join is genuinely unpredictable, and the join looks like the origin.

This is a missing measurement, not a bad model. The fix is to add a join-key overlap statistic, and it is the first thing to do next.

Part 7: Evaluation, and why it is set up this way

Chronological splits

Train on the first 60%, validate on the next 15%, test on the last 25%, in time order. A random split over a time series lets the model learn from next week to predict this week, which flatters it and would never hold in practice.

Contaminated training history

The training period contains about 3% incidents, as real history does. Most published work assumes a guaranteed-clean training set, which nobody has, because nobody knows which historical runs were healthy.

Robust normalisation

Median and interquartile range rather than mean and standard deviation, so the contaminated batches cannot define what normal means. Features that never vary while healthy, such as the schema-change flag, fall back to standard deviation, because otherwise a schema flag flipping would register as a one-sigma event, weaker than ordinary volume noise.

The metrics

- **AUROC**: separation between broken and healthy runs. Less useful here.
- **AUPRC**: the same thing but honest when only 15% of runs are broken. A model that said "healthy" every time would get 85% accuracy and be useless.
- **Best F1**: the best achievable at any threshold, an oracle upper bound.
- **F1 at an alert budget**: what an on-call rotation actually feels. You get so many alerts and no more.
- **Hit@1 / Hit@3**: how often the true culprit is ranked first, or in the top three.
- **MRR**: mean reciprocal rank. First place scores 1, second 0.5, third 0.33.

Why the tight alert budgets look bad

F1 at a 1% budget is 0.129 for almost every method, which looks terrible. It is not. That is the arithmetic ceiling: with 15% of runs broken and only 1% of runs allowed to raise an alert, you cannot recall more than a fourteenth of the incidents no matter how perfect you are. The 15% budget column, roughly one alert per incident, is the one that discriminates.

Point-adjusted F1 is reported but never headlined

Point adjustment credits you with detecting an entire incident if you catch any single moment inside it. Published re-evaluations have shown that randomly generated scores beat real models under this

metric. It is the standard in this literature and it is broken. It appears in the results labelled as such, next to the honest number, because reviewers look for it.

Part 8: How to reproduce everything

```
python3.12 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt

# Dataset, about 4 minutes
PYTHONPATH=src python -m etlguard.data.build_dataset --n-batches 8000 --out
        data/etl_v1

# Check every fault type moves the features it should
PYTHONPATH=src python scripts/validate_simulator.py

# Descriptive figures
PYTHONPATH=src python scripts/dataset_report.py --data data/etl_v1

# Pretraining, about 18 minutes on Apple Silicon
PYTHONPATH=src python -m etlguard.train.pretrain --epochs 40 --out runs/ssl

# Evaluation with all baselines
PYTHONPATH=src python -m etlguard.eval.run_eval --ckpt runs/ssl/best.pt --
        save-scores

# Two ablation rows compare against a dataset built after the timestamp
        units
# fix, so build that too. run_experiments skips those rows if it is missing.
PYTHONPATH=src python -m etlguard.data.build_dataset --n-batches 8000 --out
        data/etl_v2

# Seeds and ablations, about three hours
PYTHONPATH=src python scripts/run_experiments.py --only all

# Attribution study, error analysis, tables and figures
PYTHONPATH=src python scripts/attribution_study.py --ckpt runs/ssl/best.pt
PYTHONPATH=src python scripts/error_analysis.py --ckpt runs/ssl/best.pt
PYTHONPATH=src python scripts/results_report.py

# Demos
PYTHONPATH=src python scripts/demo.py --n 2
PYTHONPATH=src python scripts/export_demo_data.py && python scripts/
        build_web_demo.py

# Documents
bash scripts/build_report_docx.sh
bash scripts/build_litreview.sh
```

```
PYTHONPATH=src python scripts/build_slides.py

# Tests
PYTHONPATH=src python -m pytest tests/ -q
```

What lives where

Path	What it holds
`src/etlguard/simulator/`	The pipeline, the faults, the profiler, the engine
`src/etlguard/data/`	Dataset building, windowing, normalisation
`src/etlguard/models/`	The transformer, masking, the baselines
`src/etlguard/train/`	Pretraining
`src/etlguard/eval/`	Scoring, metrics, explainability, the comparison
`scripts/`	Everything runnable that is not the library
`tests/`	59 tests
`docs/`	Report, literature review, results, this file
`artifacts/`	Figures, tables, the report, the slides, the web demo

Part 9: Mistakes made, and what they cost

Kept because they are the most useful part of the project to talk about.

The peeking bug in the culprit-finder

Hiding only the task under test let the model read the answer off that task's own broken descendants. Cost: accuracy was 63.2% instead of 77.3% until it was found.

An unfair comparison, then a public correction

The structural scoring rule was first reported as worth 18.5 points. It was being compared against an uncalibrated baseline. Once both were calibrated fairly, it was worth 2.4. The report carries the correction rather than the original claim.

Timestamps a thousand times too small

`DatetimeIndex.astype("int64")` means nanoseconds in one version of pandas and microseconds in the next, so a hard-coded divisor produced timestamps in 1970 with hourly runs three seconds apart. Nothing crashed. The calendar features fed by those timestamps carried almost no signal, which meant

the calendar ablation was not testing anything. Found only because a demo script printed 1970 on screen. The experiment was redone on corrected data and the conclusion held.

Bars that rendered empty

In the web demo, the progress bars were inline elements, and inline elements ignore width. Every bar rendered as an empty track. Found by looking at the page rather than assuming it worked.

The lesson in all four

A number nobody looks at is a number nobody checks. Three of these four survived because nothing failed loudly, and were caught only when something forced them into view.